

Hyperloom

An AI agent that makes AMD Radeon GPUs faster at LLM inference — on its own. It reads the silicon's real instruction set with the assembler, finds what shipped kernels never use, writes correctness-gated HIP kernels to close the gap, and then *audits its own measurement*. Core inference runs entirely on the Radeon card. No cloud in the loop.

MEASURED ON A RADEON AI PRO R9700 · GFX1201 / RDNA4

DECODE

97% of roofline

Production int4 GEMV at 604–613 GB/s against a measured 631 GB/s DRAM roofline — memory-bound and saturating.
max_rel_err 1.1e-04 PASS

PREFILL

3.67x

int4 2:4-sparse SWMMAC vs int8 WMMA at K=8192 — 88–95% of the 3.90x raw ISA ceiling. **all gates max_abs_err = 0**

COMMS

2.46x smaller

INT6 compressed all-reduce, shared scale, exact-integer reduce — routes around the RCCL gfx1201 tuning gap. **mean rel_l2 0.0239**

THE CAPABILITY THAT MATTERS: IT PROVES ITSELF WRONG

Most speedup claims die on the measurement, not the kernel. The R9700 has a **64 MiB Infinity Cache** — any int4 benchmark whose weights fit inside it measures cache, not DRAM, and happily reports a bandwidth *above* the roofline.

So the harness prints its own working set and flags the bad reading:

N=14336 K=4096 | 31.5 MiB (CACHE-RESIDENT!) | 788 GB/s | 125% of roofline
N=65536 K=4096 | 144.0 MiB (DRAM-honest) | 609 GB/s | 97% of roofline

This caught a real error in our own submission — the decode benchmark had been reporting 122% of roofline. **An optimizer you can trust has to be able to prove itself wrong.**

The naive one-thread-per-row route looks 2.2x faster with the native instruction, and that A/B is real. But it is uncoalesced: past the cache both routes collapse to ~17 GB/s. **The win is the access pattern, not the instruction** — and we say so rather than let a judge discover it.

The same discipline runs through the stack. Faults are isolated at the layer where they actually live — silicon, driver, runtime, compiler, kernel, format, serving — rather than guessed at across all seven. And correctness is never traded for speed: a fast-but-wrong kernel **exits non-zero** before it is allowed to report a number.

REPRODUCIBLE BY A STRANGER, IN THREE COMMANDS

```
git clone https://github.com/The-Monk/rocky-hackathon
cd rocky-hackathon/kernels/decode
hipcc --offload-arch=gfx1201 -O3 decode_mmvq_iu4.hip -o d
HIP_VISIBLE_DEVICES=0 ./d # 97% of roofline, gated
```

Every number on this poster was verified from a clean clone of the public repository, on the card, on 2026-08-04. Upstream fixes contributed back: ROCm/composable_kernel#3759 (3-bug 2:4-sparse SWMMAC correctness fix, err 352 → 0) and AMD-AGI/Magpie#70.

ARCHITECTURE

CORE INFERENCE — LOCAL, ON THE RADEON GPU
Lemonade :13305 · tool-calling model · 32 GB

MISSION LOOP (reason → plan → act)

DETECT	the silicon
SCAN	ISA via llvm-mc
FIND	the unused instruction
WRITE	the HIP kernel
GATE	bit-exact vs CPU ref
MEASURE	on the card
AUDIT	the measurement itself

TOOLS
scan-isa
disasm
hipcc
magpie

MEMORY
closed
paths
are not
re-tried

ESCALATION
stuck → packet
→ AMD-hosted
→ **re-gated**
LOCALLY

Layered fault isolation
L1 silicon → L2 driver → L3 runtime
→ L4 compiler → L5 kernel → L6 format
→ L7 serving

TRACK COMPLIANCE

- Core inference executes **locally on an AMD Radeon GPU** — the agent's brain is a local model on the R9700.
- No dependence on a closed-source agent platform; the loop, tools and memory are in the repo.
- All tooling ROCm-native — hipcc, llvm-mc, rocprof/metrix, Magpie.
- Demo assets generated on the same card (kokoro narration, Flux imagery).

WHAT THE ASSEMBLER FOUND ON GFX1201 THAT THE DATASHEET DOES NOT TELL YOU

PROBED

351 builtins

Every candidate instruction compile-probed against the real target, then classified REAL / EMULATED / REJECTED. Headers and marketing both lie; the assembler does not.

ACTUALLY REAL

54 on this silicon

The rest are emulated or rejected outright — including the MX scaled-converts, which look present in the headers but are gated to other targets.

UNEXPLOITED

3 instructions

v_dot8_i32_iu4 · v_dot2_f32_f16 · v_dot4_f32_fp8 — real, fast, and unused by shipped kernels. That gap is the whole opportunity.